LAB COURSE WRITTEN ANSWERS

Object-Oriented Programming (Java) & Version Control (Git)

TP1 & TP2 SOLUTIONS

TP 1 – OBJECT-ORIENTED PROGRAMMING IN JAVA

a **Why does Account have no method setBalance? What could go wrong if balance were public?**

The `Account` class has no `setBalance()` method to protect the balance from unauthorized modifications. The balance should only be changed through controlled operations such as `deposit()` and `withdraw()`. If it were public, any part of the program could modify it directly, causing invalid values or inconsistent account data.

b **Why is nextNumber static, and why can number be final?**

`nextNumber` is static because it is shared by all instances of the `Account` class. It ensures that each account receives a unique number. The attribute `number` can be final because an account number is assigned once during object creation and should never change.

c **Why must equals and hashCode be redefined together? Why compare the numbers rather than the owners?**

`equals()` and `hashCode()` must be redefined together to respect the Java contract: if two objects are equal, they must have the same hash code. Account numbers are used because they uniquely identify accounts, whereas different accounts can have the same owner.

d **Why must super(...) be the first instruction of the constructors of the subclasses? Can SavingsAccount change balance directly? Should balance become protected?**

`super(...)` must be called first because the parent class must be initialized before the subclass. `SavingsAccount` cannot directly modify balance because it is private. The balance should remain private to preserve encapsulation. It is better to use inherited public methods such as `deposit()` rather than making the attribute protected.

e **Why is it better to redefine availableAmount() than withdraw()?**

Redefining `availableAmount()` is better because it changes the withdrawal limit without duplicating the withdrawal logic. The `withdraw()` method remains implemented once in the parent class, making the code easier to maintain and reuse.

f **In endOfMonth(), which version of monthlyFee() is executed for each account? When is this decided, and what is this mechanism called?**

The version corresponding to the actual object type is executed. For a `SavingsAccount`, the savings implementation is called; for a `CheckingAccount`, the checking implementation is called. This is decided at runtime and is called dynamic method dispatch or runtime polymorphism.

g What happens now if you write new Account("Client A", 0)? Why?

The code produces a compilation error because `Account` is now an abstract class. Abstract classes cannot be instantiated directly. An object must be created from a concrete subclass such as `SavingsAccount` or `CheckingAccount`.

h What is the difference between overriding and overloading? Give an example of each, in your code or in the Java library.

- **Overriding:** A subclass provides its own implementation of a method inherited from its parent class. Example: `CheckingAccount` overrides `availableAmount()`.
- **Overloading:** Several methods have the same name but different parameter lists. Example: `System.out.println()` has several overloaded versions that accept different data types.

i What does the annotation @Override check? What happens if you write public double monthlyfee() with it, and without it?

`@Override` checks that a method correctly overrides a method declared in a superclass or interface. If we write `monthlyfee()` instead of `monthlyFee()` with `@Override`, the compiler reports an error because Java is case-sensitive and the method name does not match. Without the annotation, the method may be treated as a new method instead of an override.

j Compare an abstract class and an interface. Could InterestBearing be an abstract class? Would it be a good idea?

An abstract class can contain attributes, constructors, concrete methods, and abstract methods. An interface mainly defines a contract that classes must implement, and it can also contain default and static methods. `InterestBearing` could be an abstract class, but an interface is a better choice because it represents a capability that different classes can implement. It also allows a class to implement multiple interfaces, while Java supports only single class inheritance.

k How can the accounts be sorted by owner without changing compareTo?

We can use a custom `Comparator<Account>` to compare accounts by their owners. This allows sorting by owner without modifying the natural ordering defined by `compareTo()`.

TP 2 – GIT QUESTIONS

a What are the three main areas of Git?

Git manages files through three primary structural areas:

1. Working Directory
The local filesystem folder containing project files where active modifications occur.

2. Staging Area
An intermediate preview area where specific changes are prepared before committing.

3. Repository
The permanent database (`.git`) storing committed history and metadata.

b What is the purpose of git status?

The `git status` command displays the current state of the working directory and staging area. It shows modified, staged, untracked, and committed files.

c Why is the target/ directory not included in the repository?

The `target/` directory contains generated files created by Maven during compilation and testing. It is excluded using `.gitignore` because these files can be regenerated and do not need to be tracked by Git.

d What are the characteristics of a good commit message?

A good commit message should be:

- Short and clear.
- Written in the imperative mood.
- Focused on one logical change.
- Descriptive of what the commit accomplishes.

e What is a branch, and what is HEAD?

- **Branch:** A movable pointer to a specific commit, allowing developers to work on different features independently.
- **HEAD:** A pointer that indicates the currently checked-out commit, usually through the current branch.

f What is the difference between a fast-forward merge and a merge commit?

- **Fast-forward merge:** Git moves the current branch pointer forward because there are no divergent commits.
- **Merge commit:** Git creates a new commit that combines two branches when their histories have diverged.

g Why can a merge conflict occur in README.md?

A merge conflict occurs when two branches modify the same lines of a file differently. Git cannot automatically determine which version should be kept, so the developer must resolve the conflict manually.

h Differences: git fetch vs git pull & git clone vs git pull

- **`git fetch` vs `git pull`:** `git fetch` downloads changes from the remote repository without integrating them into the current branch. `git pull` downloads remote changes and integrates them into the current branch, usually through a merge or rebase.
- **`git clone` vs `git pull`:** `git clone` creates a local copy of an existing remote repository. `git pull` updates an existing local repository with changes from the remote repository.

i Which Unified Process discipline and Extreme Programming (XP) practices are supported by Git?

Git mainly supports the **Configuration and Change Management** discipline of the Unified Process by tracking versions, changes, and branches.
It also supports several XP practices, such as:

- Continuous Integration.
- Collective Code Ownership.
- Refactoring.
- Small and frequent releases.